МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ

з лабораторної роботи №5
з дисципліни “Професійний Python”
на тему “Обробка помилок, файли та серіалізація”

Виконала
студентка 3 курсу
групи ФеП-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

1. Мета роботи: Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних.

Варіант 7. Імпорт та експорт системи оцінювання студентів Поля: student_id, name, discipline, grade. Перевіряти student_id, ПІБ, discipline та grade у межах 0–100. Реалізувати streaming import, tolerant/strict validation, YAML configuration з переліком дисциплін і порогом успішності, JSON export, statistics valid/invalid та error log.

2. Постановка задачі

Розробити модуль імпорту та експорту системи оцінювання студентів. Поля запису: `student_id`, `name`, `discipline`, `grade`. Перевірити цілісність `student_id`, непорожнє ПІБ, належність дисципліни до списку дозволених, та оцінку в межах 0-100. Реалізувати потоковий CSV-імпорт, JSON-експорт, YAML-конфігурацію з переліком дисциплін і порогом успішності, strict/tolerant режими, статистику (`valid`/`invalid`), логування помилок, власні винятки, context manager і атомарний запис.

3. Структура та архітектура Проєкт містить точку входу `main.py` у корені, пакет `grading_importer` із модулями `config.py`, `exceptions.py`, `models.py`, `readers.py`, `validators.py`, `services.py`, `exporters.py`, `logging_config.py`, `utils.py` (для context manager) та `main.py`. Також додано `config.yaml`, теку `data` з тестовим CSV-файлом, теку `logs` та `README.md`.

4. Схема вхідних даних

- `student_id` — ціле число.

- `name` — непорожній рядок (ПІБ студента).
- `discipline` — рядок, який обов'язково має входити до дозволеного переліку дисциплін (`allowed_disciplines`), заданого у `config.yaml`.
- `grade` — дійсне число у межах 0-100.

5. Обробка помилок і надійність Створено ієрархію винятків: `ApplicationError` $\rightarrow$ `DataError` $\rightarrow$ `DataImportError`/`RecordValidationError`/`DataExportError`, а також `ConfigurationError`. Клас `RecordValidationError` зберігає номер рядка та поле, у якому сталася помилка. Для збереження першопричини (traceback) помилок перетворення (наприклад, `ValueError` під час парсингу ID) застосовується exception chaining через конструкцію `raise ... from error`. Файли відкриваються виключно через оператор `with`, шляхи формуються за допомогою `pathlib.Path`. У режимі `tolerant mode` некоректні записи логуються (warning) та пропускаються. У `strict mode` перша ж помилка перериває імпорт. Експорт виконується атомарно: дані пишуться в тимчасовий `.tmp` файл, і лише після успішного збереження він замінює оригінальний файл.

6. Конфігурація Файл `config.yaml` задає шляхи (`input`/`output`), параметри валідації (`allowed_disciplines` для перевірки предметів та `passing_grade` для фільтрації), політику пропуску некоректних записів (`skip_invalid`) і параметри `logging` (рівень та шлях до файлу).

7. Додаткові можливості

- Власний context manager `@contextmanager logged_operation` для вимірювання часу виконання операції та логування результату.
- Потокове читання (streaming I/O) за допомогою генератора (yield) та `csv.DictReader`, щоб не завантажувати весь файл у пам'ять одразу.

- Атомарний експорт, який захищає попередні вихідні дані від пошкодження під час аварійного завершення експорту.
- Фільтрація результатів згідно з порогом успішності (`passing_grade`) перед серіалізацією.

8. Результат тестового запуску Для демонстраційного CSV-файлу програма обробила 6 записів. З них 2 валідні та 4 невалідні. Некоректні записи були виявлені та занесені до логу: оцінка понад 100, порожнє ім'я, незареєстрована дисципліна ("Chemistry") та символи замість ID. До `output/grades.json` експортовано лише валідні записи. Події імпорту, час виконання та причини відхилення записів збережено в `logs/application.log`. *(Тут додайте скриншот терміналу після запуску програми `python -m grading_importer.main`)*

9. Відповіді на контрольні запитання

Що таке exception? Об’єкт, який сигналізує про помилку або нестандартну ситуацію під час виконання. Для чого try? Для виділення коду, під час виконання якого може виникнути виняток. Для чого except? Для перехоплення й обробки конкретного типу винятку. Коли виконується else після try? Коли в блоці try не виникло винятку. Коли виконується finally? Завжди під час виходу з конструкції, зокрема після помилки. Для чого raise? Для явного створення або повторного підняття винятку. Що означає raise без аргументів? Повторно піднімає поточний виняток у блоці except. Чому не рекомендується except: pass? Він приховує помилки та ускладнює діагностику. Чому перехоплюють конкретні типи? Щоб не приховувати неочікувані помилки програмування. Що таке exception hierarchy? Ієрархія винятків, об’єднаних базовими класами. Навіщо custom exceptions? Щоб представляти помилки предметної області зрозумілими типами. Що таке exception chaining? Збереження початкової

причини під час створення нового винятку. Що означає raise ... from ...? Явно задає початковий виняток як причину нового. Перевага chaining? Traceback показує першопричину помилки. Що означає EAFP? Спробувати виконати дію й обробити виняток, якщо вона не вдалася. Що означає LBYL? Спочатку перевірити умови, а потім виконувати дію. Що таке fail-fast? Раннє завершення при виявленні критично неправильних даних або налаштувань. Де застосовувати fail-fast? Під час перевірки конфігурації та критичних передумов запуску. Що таке context manager? Об'єкт, який керує входом у ресурс і виходом із нього. Для чого with? Для безпечного керування ресурсами та їх закриття. Які методи утворюють протокол? __enter__() і __exit__(). Що робить __enter__()? Готує ресурс і повертає значення для конструкції with. Що робить __exit__()? Звільняє ресурс та може впливати на поширення винятку. Чи закриває with файл при exception? Так, вихід із контексту відбувається і при винятку. Для чого contextlib.contextmanager? Для створення context manager на основі генератора. Роль yield? Позначає точку, у якій керування передається тілу with. Що таке pathlib.Path? Об'єктний інтерфейс для роботи зі шляхами файлів і каталогів. Чому pathlib краще за конкатенацію? Він переносний і правильно формує шляхи для різних ОС. Навіщо encoding=utf-8? Для передбачуваного читання й запису Unicode-тексту. Що таке CSV? Текстовий формат табличних даних із розділеними полями. Для чого csv.reader? Для читання CSV як послідовності списків полів. Для чого csv.DictReader? Для читання рядків CSV як словників за назвами стовпців. Чим зручний DictReader? Дозволяє звертатися до значень за назвами полів. Для чого DictWriter? Для запису словників у CSV. Що таке JSON? Текстовий формат обміну структурованими даними. Що таке serialization? Перетворення об'єкта на формат для зберігання чи передавання. Що таке deserialization? Відновлення структури даних із серіалізованого представлення. json.dump()

проти json.dumps()? dump записує у файловий об’єкт, dumps повертає рядок. json.load() проти json.loads()? load читає з файлового об’єкта, loads розбирає рядок. Що таке JSONDecodeError? Виняток для синтаксично некоректного JSON. Для чого YAML? Для зручних для людини конфігурацій і структурованих даних. Чому yaml.safe_load()? Він не дозволяє конструювати довільні Python-об’єкти з недовіреного YAML. Типові сценарії JSON? API, обмін даними та експорт. Типові сценарії YAML? Конфігурації, які зручно редагувати вручну. Що таке configuration file? Зовнішній файл параметрів поведінки програми. Чому не hard-code? Налаштування можна змінювати без редагування Python-коду. Що перевіряти в конфігурації? Типи, обов’язкові ключі, допустимі значення, шляхи та взаємні обмеження. Що таке logging? Запис структурованих повідомлень про роботу програми. Logging проти print()? Logging має рівні, handlers, форматування та збереження журналу. Які є рівні logging? DEBUG, INFO, WARNING, ERROR, CRITICAL. Для чого logger.info()? Для звичайних інформаційних подій. Для чого logger.warning()? Для проблем, які не обов’язково зупиняють програму. Для чого logger.error()? Для помилок, що заважають виконанню операції. Для чого logger.exception()? Для запису помилки разом із traceback усередині except. Що містить traceback? Послідовність викликів, що привела до винятку. Чому не логувати secrets? Щоб не розкривати паролі, токени та конфіденційні дані. Що таке streaming I/O? Послідовна обробка даних частинами, без повного завантаження в пам’ять. Перевага streaming? Менше споживання пам’яті на великих файлах. Що таке eager loading? Завантаження всіх записів у пам’ять одразу. Чому list(reader) небажаний? Він зберігає всі рядки в RAM. Що таке strict mode? Перша невалідна запись перериває обробку. Що таке tolerant mode? Невалідні записи логуються та пропускаються. Коли strict mode? Коли кожен запис обов’язковий і помилки не можна ігнорувати. Коли tolerant mode? Коли

корисно імпортувати коректну частину набору. Що таке atomic write? Запис у тимчасовий файл із заміною цільового лише після успіху. Навіщо temporary file? Щоб не пошкодити попередній результат при невдалому експорті. Що станеться при звичайному записі й аварії? Цільовий файл може залишитися неповним. Навіщо видаляти тимчасовий файл? Щоб не залишати сміття та неповний результат. Що таке JSON Lines? Формат, де кожен рядок є окремим JSON-об’єктом. Чому JSON Lines зручний для streaming? Його можна записувати й читати рядок за рядком.

Висновок

У лабораторній роботі реалізовано надійний модуль імпорту й експорту системи оцінювання студентів для варіанта 7. Програма ефективно перевіряє зовнішні дані на цілісність, підтримує потокове читання, що зменшує споживання оперативної пам'яті, та має гнучку YAML-конфігурацію. Реалізовано ієрархію власних винятків, логування, строгий і толерантний режими обробки помилок. Експорт даних у формат JSON відбувається безпечно завдяки атомарному запису, що гарантує збереження попередніх даних у разі виникнення критичної помилки.